

Signals and Systems

A Review of Mathematical Modeling, State Machines, and Linear Systems

Shashwat Saxena
IIIT Vadodara

June 2026

Contents

1	Introduction	4
2	Signals	4
2.1	Continuous-Time and Discrete-Time Signals	4
2.2	Real-World Signal Models	4
2.2.1	Audio Signals	4
2.2.2	Images	5
2.2.3	Video	5
2.2.4	Sequences	5
3	Systems	5
3.1	Memoryless vs. Dynamic Systems	6
3.2	Mathematical Models of Systems	6
3.2.1	Continuous-Time: Differential Equations	6
3.2.2	Discrete-Time: Difference Equations	6
3.3	System Composition	6
3.3.1	Cascade (Series) Composition	6
3.3.2	Parallel Composition	7

3.3.3	Feedback Composition	7
4	Linear Time-Invariant (LTI) Systems	7
4.1	Linearity	7
4.2	Time Invariance	7
4.3	SISO and MIMO Systems	8
5	State Spaces and State Machines	8
5.1	State Space	8
5.2	Finite State Machines (FSM)	8
5.3	Feedback and Fixed Points	8
6	Linear State-Space Systems	9
6.1	The General Formulation	9
6.2	Solution to the State Equation	9
6.3	Stability Analysis via Eigenvalues	9
7	Application I: The Discrete-Time Harmonic Oscillator	10
7.1	Motivation	10
7.2	The Rotation Matrix	10
7.3	State-Space Realization	10
7.4	Output	11
7.5	MATLAB Simulation	11
8	Application II: The 2-Tap Moving Average Low-Pass Filter	11
8.1	Motivation	11
8.2	The Difference Equation	11
8.3	State-Space Realization	11
8.4	Stability via Eigenvalues	12

8.5	Frequency Response	12
8.6	Block Diagram	13
9	Other MATLAB Models	13
9.1	Basic Discrete Signals	13
9.2	Amplitude Modulation (Communication Channel Simulator)	13
10	Representation of Signals as Vector Spaces	14

1 Introduction

Signals are functions that map independent physical quantities (such as time or space) onto dependent physical quantities (such as air pressure or light intensity):

$$\text{signal} : \text{independentPhysicalQuantity} \rightarrow \text{dependentPhysicalQuantity}$$

Systems are functions that transform or modify signals, mapping an input signal space onto an output signal space:

$$\text{system} : \text{inputSignalSpace} \rightarrow \text{outputSignalSpace}$$

Together, signals and systems form the mathematical foundation of all communication, control, and signal processing theory.

2 Signals

2.1 Continuous-Time and Discrete-Time Signals

A **continuous-time signal** is defined for every value of time within an interval and is modeled as:

$$x : \mathbb{R} \rightarrow \mathbb{R}$$

Examples include audio pressure waves, temperature profiles, and electrical voltages.

A **discrete-time signal** is defined only at specific instants, indexed by an integer n :

$$x : \mathbb{Z} \rightarrow \mathbb{R}$$

Examples include sampled audio, digital images, and data sequences.

2.2 Real-World Signal Models

2.2.1 Audio Signals

An audio signal maps time onto the relative change in air pressure. Because sound is produced through compression and rarefaction of air, the time average of any audio signal is zero:

$$\text{audio} : \mathbb{R} \rightarrow \mathbb{R}, \quad \frac{1}{T} \int_0^T x(t) dt = 0$$

Signals are normalised by subtracting the ambient (DC) pressure, leaving only the fluctuating component.

2.2.2 Images

A greyscale image maps a 2D spatial domain onto a scalar intensity:

$$\text{image}_{\text{greyscale}} : [0, H] \times [0, W] \rightarrow [0, B_{\text{max}}]$$

where B_{max} is the maximum brightness (white), and 0 corresponds to black. In the mathematical vector-space formulation:

$$\text{image}_{\text{greyscale}} : \mathbb{R}^2 \rightarrow \mathbb{R}$$

For a colour image, the co-domain becomes three-dimensional (red, green, blue channels):

$$\text{image}_{\text{colour}} : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

2.2.3 Video

A video is a sequence of images (frames) displayed at a fixed frame rate. Formally, the n^{th} frame is a function from spatial coordinates to intensity:

$$\text{video} : \mathbb{N} \rightarrow (\mathbb{R}^2 \rightarrow \mathbb{R})$$

The frame rate (frames per second) determines perceived smoothness.

2.2.4 Sequences

A sequence signal maps the natural numbers onto some entity:

$$\text{sequence} : \mathbb{N} \rightarrow \mathcal{E}$$

Two common types are:

- **Data sequences** — ordered records (e.g. stock prices, sensor logs).
- **Event streams** — timestamped occurrences (e.g. network packets, keyboard input).

3 Systems

A system $\mathcal{H}$ maps an input signal to an output signal:

$$\mathcal{H} : \text{inputSignalSpace} \rightarrow \text{outputSignalSpace}$$

3.1 Memoryless vs. Dynamic Systems

A **memoryless** system's output depends only on the present input value:

$$y(t) = 2x(t)$$

A **dynamic** system's output depends on past inputs or past outputs, requiring internal memory:

$$y(n) = x(n) + x(n-1)$$

3.2 Mathematical Models of Systems

3.2.1 Continuous-Time: Differential Equations

Physical systems (RC circuits, thermal systems) are commonly described by ordinary differential equations. A general first-order system:

$$\frac{dy(t)}{dt} + ay(t) = bx(t)$$

3.2.2 Discrete-Time: Difference Equations

Digital filters and sampled-data systems are described by difference equations. A first-order difference equation:

$$y(n+1) = ay(n) + bx(n)$$

3.3 System Composition

Complex systems are built by composing simpler subsystems.

3.3.1 Cascade (Series) Composition

The output of $\mathcal{H}_1$ feeds into $\mathcal{H}_2$:

$$w(n) = \mathcal{H}_1\{x(n)\}, \quad y(n) = \mathcal{H}_2\{w(n)\} = \mathcal{H}_2\{\mathcal{H}_1\{x(n)\}\}$$

For LTI systems, cascade order is commutative: $\mathcal{H}_1 \circ \mathcal{H}_2 = \mathcal{H}_2 \circ \mathcal{H}_1$, and in terms of impulse responses:

$$h(n) = h_1(n) * h_2(n)$$

3.3.2 Parallel Composition

Both systems receive the same input; outputs are summed:

$$y(n) = \mathcal{H}_1\{x(n)\} + \mathcal{H}_2\{x(n)\}$$

For LTI systems, the equivalent impulse response is $h(n) = h_1(n) + h_2(n)$.

3.3.3 Feedback Composition

The output is routed back and subtracted from the input:

$$e(n) = r(n) - \mathcal{H}_2\{y(n)\}, \quad y(n) = \mathcal{H}_1\{e(n)\}$$

Solving for $y(n)$ gives the closed-loop transfer function:

$$y(n) = \frac{\mathcal{H}_1}{1 + \mathcal{H}_1\mathcal{H}_2}\{r(n)\}$$

Feedback introduces memory, enables self-regulation, and can stabilise otherwise unstable systems.

4 Linear Time-Invariant (LTI) Systems

4.1 Linearity

A system $\mathcal{H}$ is **linear** if it satisfies the superposition principle, which combines two properties:

Homogeneity: $\mathcal{H}\{ax\} = a\mathcal{H}\{x\}$

Additivity: $\mathcal{H}\{u + v\} = \mathcal{H}\{u\} + \mathcal{H}\{v\}$

Combined, for arbitrary scalars a, b :

$$\mathcal{H}\{au + bv\} = a\mathcal{H}\{u\} + b\mathcal{H}\{v\}$$

4.2 Time Invariance

A system is **time-invariant** if its behaviour does not change with time. Equivalently, delaying the input by n_0 delays the output by the same amount:

$$x(n - n_0) \xrightarrow{\mathcal{H}} y(n - n_0)$$

4.3 SISO and MIMO Systems

A **SISO** (Single-Input Single-Output) system has scalar input and output signals. Examples: an audio amplifier, an RC low-pass filter.

A **MIMO** (Multiple-Input Multiple-Output) system operates on vector-valued signals, commonly modelled as:

$$\mathbf{y}(t) = H \mathbf{x}(t), \quad H \in \mathbb{R}^{q \times m}$$

Examples: multi-antenna wireless communication, aircraft control systems.

5 State Spaces and State Machines

5.1 State Space

The **state space** of a system is the minimum set of information required to completely determine all future outputs given any future input. For a Rubik's Cube, the state space is the set of all possible tile arrangements ($\approx 4.3 \times 10^{19}$ states).

5.2 Finite State Machines (FSM)

A Finite State Machine is formally a five-tuple:

$$(\mathcal{S}, \mathcal{X}, \mathcal{Y}, f_{\text{update}}, s_0)$$

where $\mathcal{S}$ is the set of states, $\mathcal{X}$ the input alphabet, $\mathcal{Y}$ the output alphabet, $f_{\text{update}} : \mathcal{S} \times \mathcal{X} \rightarrow \mathcal{S}$ the transition function, and $s_0 \in \mathcal{S}$ the initial state.

A system is **deterministic** if f_{update} maps each (state, input) pair to exactly one next state. It is **non-deterministic** if multiple next states are possible.

5.3 Feedback and Fixed Points

When a system's output feeds back into its input, steady-state behaviour is characterised by **fixed points**: values x^* satisfying:

$$f(x^*) = x^*$$

Bisimulation is a stronger notion of equivalence between two systems A and B : both must simulate each other, meaning they are behaviourally indistinguishable from the outside.

6 Linear State-Space Systems

6.1 The General Formulation

For a discrete-time linear system, the state, input, and output are all vectors in real vector spaces:

$$\mathbf{s}(n) \in \mathbb{R}^p, \quad \mathbf{x}(n) \in \mathbb{R}^m, \quad \mathbf{y}(n) \in \mathbb{R}^q$$

Since the system is linear, the update and output functions must be linear maps, represented by matrices. The **state-space equations** are:

$$\mathbf{s}(n+1) = A\mathbf{s}(n) + B\mathbf{x}(n) \tag{1}$$

$$\mathbf{y}(n) = C\mathbf{s}(n) + D\mathbf{x}(n) \tag{2}$$

The matrix dimensions must be consistent:

Matrix	Dimension	Role
A	$p \times p$	maps state to next state (system dynamics)
B	$p \times m$	maps input into the state
C	$q \times p$	maps state to output
D	$q \times m$	direct feedthrough from input to output

6.2 Solution to the State Equation

For an autonomous system (no input, $\mathbf{x}(n) = \mathbf{0}$), equation (1) reduces to $\mathbf{s}(n+1) = A\mathbf{s}(n)$. Applying this recursively:

$$\mathbf{s}(n) = A^n \mathbf{s}(0)$$

The long-term behaviour is entirely determined by the **eigenvalues** of A .

6.3 Stability Analysis via Eigenvalues

Let λ_i denote the eigenvalues of A . For a scalar system ($p = 1$), A is simply a scalar a , and $s(n) = a^n s(0)$. The stability conditions are:

$$|\lambda_i| < 1 \quad \forall i \implies \text{asymptotically stable (response decays)}$$

$$|\lambda_i| = 1 \quad \forall i \implies \text{marginally stable (response constant amplitude)}$$

$$\exists i : |\lambda_i| > 1 \implies \text{unstable (response grows without bound)}$$

7 Application I: The Discrete-Time Harmonic Oscillator

7.1 Motivation

The goal is to construct a purely autonomous discrete-time system whose state traces a circle in $\mathbb{R}^2$ indefinitely — a **discrete-time harmonic oscillator**. The key insight is that repeated application of a 2D rotation matrix rotates a vector by the same angle at each step, tracing a circle.

7.2 The Rotation Matrix

A rotation by angle ω (radians) in $\mathbb{R}^2$ is represented by:

$$A = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix}$$

Properties of A :

- $\det(A) = \cos^2 \omega + \sin^2 \omega = 1$ — area-preserving.
- A is orthogonal: $A^{-1} = A^\top$ — the inverse is just the transpose.
- Eigenvalues: $\lambda = e^{\pm j\omega}$, so $|\lambda| = 1$ — **marginally stable**.

Since all eigenvalues lie on the unit circle, the oscillation neither grows nor decays.

7.3 State-Space Realization

The state vector stores both coordinates of the rotating point:

$$\mathbf{s}(n) = \begin{bmatrix} s_1(n) \\ s_2(n) \end{bmatrix} = \begin{bmatrix} \cos(n\omega) \\ \sin(n\omega) \end{bmatrix}, \quad \mathbf{s}(0) = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

The state update equation expands to:

$$\begin{bmatrix} s_1(n+1) \\ s_2(n+1) \end{bmatrix} = \begin{bmatrix} \cos \omega & -\sin \omega \\ \sin \omega & \cos \omega \end{bmatrix} \begin{bmatrix} s_1(n) \\ s_2(n) \end{bmatrix}$$

which gives the scalar recurrences:

$$\begin{aligned} s_1(n+1) &= s_1(n) \cos \omega - s_2(n) \sin \omega \\ s_2(n+1) &= s_1(n) \sin \omega + s_2(n) \cos \omega \end{aligned}$$

7.4 Output

Taking $C = \begin{bmatrix} 1 & 0 \end{bmatrix}$ extracts the first component:

$$y(n) = C \mathbf{s}(n) = s_1(n) = \cos(n\omega)$$

The system output is a pure discrete-time cosine with no input required — all energy comes from the initial condition $\mathbf{s}(0)$.

7.5 MATLAB Simulation

In MATLAB, using $\omega = \pi/60$ rad/step and $n = 1000$ steps with $\mathbf{s}(0) = [0 \ 1]^\top$, the phase portrait (plot of s_2 vs. s_1) traces a perfect circle, confirming marginal stability.

8 Application II: The 2-Tap Moving Average Low-Pass Filter

8.1 Motivation

Consider a signal corrupted by high-frequency noise. The moving average filter recovers the underlying signal by averaging consecutive samples — high-frequency oscillations cancel, while the slowly-varying signal survives.

8.2 The Difference Equation

The 2-tap moving average is:

$$y(n) = \frac{x(n) + x(n-1)}{2}$$

This is an **FIR** (Finite Impulse Response) filter: its output depends only on a finite window of past inputs, with no feedback from past outputs.

In the MATLAB model, the input signal is:

$$x(n) = \sin(2\pi \cdot 5 \cdot t_n) + 0.5 \sin(2\pi \cdot 440 \cdot t_n), \quad F_s = 1000 \text{ Hz}$$

a 5 Hz base signal corrupted by 440 Hz noise at half amplitude.

8.3 State-Space Realization

Choose the state $s(n) = x(n-1)$ (the one past sample required). Then $p = m = q = 1$, so all matrices reduce to scalars. The ABCD parameters are:

$$A = 0, \quad B = 1, \quad C = \frac{1}{2}, \quad D = \frac{1}{2}$$

Substituting into equations (1)–(2):

$$s(n+1) = x(n) \quad (\text{state stores current input})$$

$$y(n) = \frac{1}{2}x(n-1) + \frac{1}{2}x(n) \quad (\text{output is the mean of current and previous input})$$

Interpretation of each parameter:

- $A = 0$: the state does not feed back into itself — there are no autonomous dynamics.
- $B = 1$: the current input is stored directly into the state.
- $C = D = \frac{1}{2}$: the output is the average of the current and previous input.

Since $A = 0$, the zero-input response is $s(n) = 0^n \cdot s(0) = 0$ for all $n \geq 1$. The filter is unconditionally stable and entirely input-driven.

8.4 Stability via Eigenvalues

The single eigenvalue of A is $\lambda = 0$, which satisfies $|\lambda| = 0 < 1$. The system is therefore **asymptotically stable** — in the absence of input, the state decays to zero in one step.

8.5 Frequency Response

To understand which frequencies the filter passes or rejects, apply a complex exponential input $x(n) = e^{j\omega n}$:

$$y(n) = \frac{1}{2}e^{j\omega n} + \frac{1}{2}e^{j\omega(n-1)} = e^{j\omega n} \cdot \frac{1}{2}(1 + e^{-j\omega})$$

The **frequency response** (eigenvalue of the LTI system for complex exponential input) is:

$$H(e^{j\omega}) = \frac{1}{2}(1 + e^{-j\omega})$$

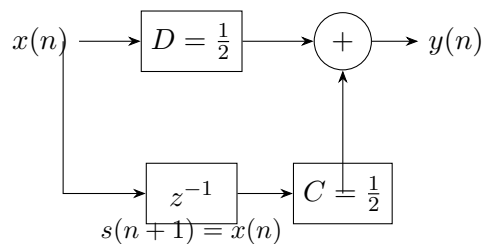
Its magnitude is obtained by factoring out $e^{-j\omega/2}$:

$$|H(e^{j\omega})| = \left| \frac{e^{j\omega/2} + e^{-j\omega/2}}{2} \right| = \left| \cos \frac{\omega}{2} \right|$$

This result directly explains the filter's behaviour:

- $\omega = 0$ (DC): $|H| = |\cos 0| = 1$ — DC component passes through unchanged.
- $\omega = \pi$ (Nyquist frequency, highest possible): $|H| = |\cos(\pi/2)| = 0$ — completely rejected.
- The 440 Hz noise at $F_s = 1000$ Hz corresponds to normalized frequency $\omega = 2\pi \cdot 440/1000 = 0.88\pi$, which is near π and thus strongly attenuated.

8.6 Block Diagram



There is no feedback path from output to input, confirming the FIR (no-IIR-poles) structure. The z^{-1} block stores $x(n)$ and outputs $x(n-1)$ on the next step.

9 Other MATLAB Models

9.1 Basic Discrete Signals

The following elementary discrete-time signals were generated and plotted in MATLAB:

- **Unit impulse:** $\delta(n) = \begin{cases} 1 & n = 0 \\ 0 & n \neq 0 \end{cases}$ — the fundamental building block of LTI analysis.
- **Unit step:** $u(n) = \begin{cases} 1 & n \geq 0 \\ 0 & n < 0 \end{cases}$
- **Unit ramp:** $r(n) = n u(n)$
- **Square pulse:** $p(n) = u(n) - u(n - N)$ for a pulse of width N .
- **Exponential decay:** $x(n) = a^n u(n)$, $|a| < 1$

Any discrete-time signal can be expressed as a weighted sum of shifted unit impulses:

$$x(n) = \sum_{k=-\infty}^{\infty} x(k) \delta(n - k)$$

This decomposition is the basis of convolution and impulse response analysis.

9.2 Amplitude Modulation (Communication Channel Simulator)

A software AM modulator was built combining a 50 Hz baseband signal with a 220 Hz carrier wave. If the message signal is $m(t) = \cos(2\pi f_m t)$ and the carrier is $c(t) = \cos(2\pi f_c t)$, the AM signal is:

$$x_{\text{AM}}(t) = [1 + k_a m(t)] \cos(2\pi f_c t)$$

where k_a is the amplitude sensitivity. This model demonstrates signal preparation for transmission and visualises the modulation envelope in the time domain.

10 Representation of Signals as Vector Spaces

Every signal space can be given a vector-space structure. For instance, the set of square-integrable functions $L^2(\mathbb{R})$ forms a Hilbert space with inner product:

$$\langle x, y \rangle = \int_{-\infty}^{\infty} x(t) \overline{y(t)} dt$$

This perspective unifies signal processing with linear algebra: filters are linear operators, frequency components are basis vectors (complex exponentials $\{e^{j\omega t}\}$), and the Fourier transform is a change of basis.

For the discrete-time signals studied in this work, the relevant space is $\ell^2(\mathbb{Z})$ — sequences with finite energy:

$$\|x\|^2 = \sum_{n=-\infty}^{\infty} |x(n)|^2 < \infty$$

The frequency response $H(e^{j\omega})$ is the eigenvalue of an LTI system corresponding to the eigenfunction $e^{j\omega n}$, making eigenvalue analysis central to filter design.